

Person 4 -- Wow-Factor Layer and the Vignette Dataset

NHAA Trauma Triage System -- Standalone Build Specification

How to Use This Document

This is a complete, standalone build specification. Build every phase directly, in order, without waiting for or requiring live coordination with anyone else. Each module below is fully self-contained and independently testable.

Your Role

You build the Silent SOS module, the Referral Booking mock, the Agent Wellbeing signal, the Accessibility Bridge backend logic, and -- starting immediately, in Phase 1 -- the vignette dataset for the two dimensions with no public data anywhere: `fear_of_retaliation` and `intimidation`.

Phase 1 -- Write the Vignette Dataset (Do This First)

Write 15-20 short vignettes (2-4 sentences each) per dimension, for **fear_of_retaliation** and **intimidation**, in Hindi, English, and Hinglish, across four severity levels matching Low/Moderate/High/Critical. Ground the language in general, publicly documented patterns of caste-based intimidation -- do not use any real person's name or identifying case detail. Example patterns to draw from (write your own full vignettes around ideas like these): being told not to report an incident, a perpetrator's associates visiting the caller's home afterward, fear of losing work or housing if the complaint proceeds, an authority figure discouraging the complaint.

Save these as `keywords/fear_of_retaliation.json` and `keywords/intimidation.json`, each entry formatted as:

```
    {"phrase": "wo phir aayenge agar main report karti hoon",  
     "language": "hinglish", "severity": "high"}
```

This file format is designed to be dropped directly into another team member's `keywords/` folder at merge time with no changes needed -- you do not need to hand this to anyone or coordinate; simply have the finished files ready in your own project folder.

Phase 2 -- Build the Silent SOS

Module

Build a standalone Python function `detect_duress_code(dtmf_sequence: str) -> bool` that returns True if a specific agreed sequence (use "9631" as the default -- a sequence unlikely to be pressed by accident during normal call navigation) appears in a stream of keypad presses. Build a small simulated DTMF input generator for testing (a Python list of digit-presses with random other digits mixed in, plus your target sequence embedded once) and confirm your function correctly detects the embedded sequence and does not false-positive on the random digits around it.

Build a second function `apply_duress_mode(current_bucket: str) -> dict` that returns an object like `{"bucket": "critical", "duress_active": true, "agent_script_mode": "yes_no_only"}` -- this represents the system's response to a detected duress signal, and it deliberately does **not** include any field that would let this bypass the senior-approval gate; it only raises urgency and changes the suggested question format.

Phase 3 -- Build the Referral Booking Mock and Agent Wellbeing Signal

Referral booking -- create a file `mock_directory.json` containing 10 fake counseling-center/legal-aid entries, each with name, type (counseling/legal_aid), district, and a list of available time slots (fake dates/times). Build a function `find_and_book_slot(district, referral_type, case_summary: dict = None) -> dict` that searches this file, returns the earliest available slot, marks it as booked so it won't be offered twice, and -- if `case_summary` is provided -- includes it in the returned booking confirmation object as `"handoff_summary": case_summary`, so the receiving counsellor gets the risk-dimension context in advance rather than the victim needing to re-explain from nothing (this is the specific requirement in Section 2.22 of the solution document: "the receiving counsellor or legal-aid worker is given the case's risk-dimension summary in advance"). Test: call it twice for the same district/type and confirm it returns two different slots, and confirm the `handoff_summary` field is present and correctly populated when a `case_summary` dict is passed in.

Agent wellbeing -- look up the ProQOL-5 instrument directly online (search "ProQOL-5 questionnaire items") and note its three subscale categories: Compassion Satisfaction, Burnout, and Secondary Traumatic Stress. Build a function `wellbeing_signal(response_times: list, avg_response_length: list) -> dict` that takes a simulated list of an agent's response times and message lengths across a mock shift (write fake data: 20 numbers trending slower and shorter over the "shift" to simulate fatigue) and returns a flag `{"category": "burnout", "concern_level": "moderate"}` when response times trend more than 30% slower than the first five calls' average. This is a simple heuristic, not a trained model -- document in your code comments that this is a starting heuristic mapped to ProQOL's Burnout category, intended for future validation against the real instrument.

Phase 4 -- Accessibility Bridge Backend

Write a function `score_text_only_input(transcript: str) -> dict` that takes plain text (no audio) and returns the same risk-vector shape as the full pipeline, using the same keyword-matching logic pattern described in Person 1's document (scan for phrases from the keywords/JSON files, score by match count). Since you do not have Person 1's real `risk_scorer.py` module, write your own simplified standalone version of this exact same logic for this phase -- it only needs to work on text, so it is a smaller version of what Person 1 is separately building for the full voice pipeline. Test it against one of your own Phase 1 vignettes and confirm it correctly scores the relevant dimension.

Phase 5 -- Package for Merge

Combine your five deliverables (vignette JSON files, `detect_duress_code`, `apply_duress_mode`, `find_and_book_slot`, `wellbeing_signal`, `score_text_only_input`) into one Python module `nhaa_wow_factors.py` with clear, separately-callable functions. Write a `README_wow_factors.md` listing each function, what it takes as input, what it returns, and one example call for each -- written so that whoever merges all five people's work at the end can import and use your functions without reading your internal code.